When Does Prefill–Decode Disaggregation Pay Off Without RDMA?

A Measurement Study over TCP on Cloud GPU Nodes

Sunghyun Jun
Independent Researcher
[contact email]

[draft, 2026-09-30]

Abstract

Most work on prefill–decode (PD) disaggregated serving assumes that GPUs are linked by RDMA or NVLink. Rented cloud GPUs often have neither, and the KV cache must cross ordinary Ethernet over TCP. We ran vLLM with NIXL/UCX over TCP on three AWS single-GPU instance types and served Gemma 4 26B (mixture of experts) and 31B (dense) on one to three nodes, 389 measurement runs in total, of which 256 passed a data-quality audit. The time that disaggregation adds to the first token is almost entirely KV-cache transfer time, and a simple formula predicted it before we measured it. The effective transfer rate stayed at 1.3–1.7 times the cloud's per-flow bandwidth cap. System-wide latency percentiles looked healthy while many individual requests stalled mid-response, and those stalls decide whether disaggregation pays off. When a service must keep every gap between tokens under 50–75 ms, disaggregation wins: for 26B with 4K-token inputs it raised the load each GPU can serve within the SLO by 1.5–4× and cut the cost per SLO-meeting token by about 3×, and for the dense 31B model in chat only disaggregated configurations met the SLO. For chat-length inputs it brought no gain, and when gaps of 100 ms or more are acceptable the difference mostly disappears. Heavier weights (FP8 or BF16 rather than NVFP4) made the gain larger. We distill the results into a checklist that decides in advance whether disaggregation is worth deploying, and we release the measurement harness and per-request raw data.

1 Introduction

Serving a request to a large language model takes two phases: *prefill*, which processes the whole input at once, and *decode*, which produces output tokens one at a time. When both run on the same GPU, a long prefill blocks the decode steps of requests that are already streaming, and their token stream stalls. Disaggregated serving assigns the two phases to different GPUs to remove this interference, and ships the KV cache produced by prefill to the decode GPU [1–3].

The price of disaggregation is the KV-cache transfer, and prior systems put it on fast links. Splitwise uses InfiniBand between nodes [2] and Mooncake uses an RDMA network [3]. DistServe, when cross-node bandwidth is limited, places prefill and decode on the same node so that the KV cache moves only over NVLink [1]; that option does not exist when a node has a single GPU. Real inference fleets are more varied: the range of inference GPUs has grown, many cloud instances carry one GPU each, and on rented infrastructure the network is not ours to choose. Public documentation is mostly discouraging about this setting. The llm-d documentation calls TCP transfer

"extremely slow" and "targeted for local development" [4], and the NVIDIA Dynamo documentation warns of a 200–500× slower time to first token (TTFT) without RDMA and recommends aggregated serving in that case [5]. Measurements that show how slow it is, and under which conditions, are scarce.

Work on disaggregation over slow links has started to appear. Prefill-as-a-Service sends KV caches between datacenters over commodity Ethernet [6], PDD builds cross-datacenter disaggregation over wide-area TCP [7], and the llm-d project measured KV transfer libraries over TCP [8]. These are purpose-built systems running between large clusters. Our goal is different: we take the published design as implemented in open-source tools (vLLM and NIXL) and measure when disaggregation pays off across TCP links between rented cloud GPU nodes.

We study settings where GPUs lack a fast interconnect and the KV cache must cross an ordinary network, and we measure the case of TCP between nodes. We ask four research questions (RQs):

- **RQ1.** Can disaggregation raise the throughput that meets a service-level objective (SLO goodput) even over slow TCP links?
- **RQ2.** What determines the TTFT overhead of disaggregation?
- **RQ3.** Under which conditions does disaggregation win?
- **RQ4.** When scaling out, how should the prefill-to-decode ratio be chosen?

Our contributions are:

- a per-request SLO evaluation, and an analysis that re-judges the same runs under a grid of SLO thresholds to show how much each conclusion depends on the chosen SLO;
- evidence that the TTFT overhead equals the KV transfer time, and a model of it that we validated by predicting results before measuring them;
- the boundaries at which disaggregation pays off, as a function of input length, weight precision, model size, and tolerance for mid-response stalls;
- a ratio rule for three-node deployments: grow the side that is the bottleneck;
- the measurement harness and per-request raw data, including settings and techniques that did not help.

2 Background and Related Work

Disaggregated serving. DistServe splits prefill and decode, places resources for each phase against its latency target, and uses per-GPU goodput at an SLO attainment goal (e.g., 90%) as its metric [1]; we follow the same metric. Its testbed had 25 Gbps between nodes, and in that regime it confined KV transfer to intra-node NVLink. Splitwise observes that the two phases have different hardware needs, runs them on different machine types, and sends the KV cache over InfiniBand, transferring it layer by layer for long prompts to overlap it with computation [2]. Mooncake builds a KV-cache-centric disaggregated architecture and serves Kimi with it [3]. TetriInfer [9], DéjàVu [10], and P/D-Serve [11] belong to the same family. All of them assume a fast interconnect such as RDMA or NVLink, or emulate one in their evaluation.

Improving colocation. Sarathi-Serve splits prefills into chunks that are batched together with decodes (chunked prefill) to limit interference [12]; our replica configurations use chunked prefill as well. Sarathi-Serve defines its inter-token SLO as 5× (strict) or 25× (relaxed) the decode-step time

without interference. semi-PD disaggregates only compute (at the SM level) inside one GPU and keeps a unified KV storage [13].

KV transfer layer. vLLM [14] moves the KV cache with NIXL [15], which wraps several transports including UCX [16]; without RDMA, UCX falls back to TCP. CacheGen compresses the KV cache to reduce network traffic [17].

Disaggregation over slow links. Prefill-as-a-Service exploits the small KV caches of hybrid-attention models to send them between datacenters over commodity Ethernet [6]. PDD disaggregates across two clusters connected by wide-area TCP and adds relay decoding to overlap transfer with computation [7]. The llm-d project reports GPU-to-GPU transfer between two H100 nodes over 100 Gbps TCP at 0.55 GB/s with UCX and about 4.7–4.9 GB/s with UCCL [8]. The NVIDIA Dynamo documentation reports a TTFT of about 98 s with TCP on AWS p5.48xlarge but does not state the model or input length [5]. Mitra et al. find disaggregation most effective for prefill-heavy traffic and larger models [18], and the vLLM documentation states that disaggregation does not improve throughput [19].

Where this work sits. The systems above design new infrastructure for large clusters. We instead measure stock vLLM on rented GPU nodes, and we add a model that predicts the transfer time and a checklist for deciding whether to disaggregate. We did not find a study of the same setting, but our search was not exhaustive, so we do not claim to be the first.

3 Methodology

3.1 Environment

We used three GPU instance types in one availability zone of AWS us-east-1, each with a single GPU (Table 1). Within the zone a single TCP flow was capped at about 4.97 Gbps, matching the documented 5 Gbps single-flow limit [20]. Inside a cluster placement group (an AWS feature that places instances close together in the network) the cap rose to 9.54 Gbps (documented maximum 10 Gbps). The nominal 50 Gbps of g7e was reached only by aggregating eight flows (49.72 Gbps). Load generation and request routing ran on a separate CPU instance.

Instance	GPU	GPU memory	Main experiments
g7.2xlarge	RTX PRO 4500	32 GB	two-node input-length comparison, transfer tuning
g6e.2xlarge	L40S	48 GB	cross-GPU comparison, L40S pair
g7e.2xlarge	RTX PRO 6000	96 GB	cross-GPU comparison, placement group, 2–3-node scaling, repeats

Table 1: GPU instances. Costs use the g7e on-demand price of $3.363 per hour.

We used vLLM v0.30.0 with NIXL's pull-based connector. We raised UCX's TCP segment size from the default 8 KB to 256 KB (Section 4.2). The chunked-prefill size was 2,560 tokens and GPU memory utilization 0.90.

3.2 Models and KV size

We served two Gemma 4 models [21]. 26B-A4B is a mixture-of-experts model that activates 3.8B of its 25.2B parameters per token; 31B is dense (30.7B parameters). Both interleave sliding-window

and global attention. We mainly used NVIDIA's NVFP4 (4-bit floating point) checkpoints, and FP8 (weight-only) and BF16 checkpoints for comparison. The NVFP4 checkpoints store the KV cache in FP8 (per their quantization configuration files [22]); the FP8 and BF16 checkpoints store it in BF16. Hence one 4K-token request of the same 26B model transfers 140 MB with NVFP4 (about 34 KB per input token) but 280 MB with FP8. 31B NVFP4 transfers 560 MB (about 137 KB per input token).

3.3 Configurations

We compared two approaches at equal GPU count. In the *replica* configuration each node runs a server that performs both prefill and decode, and requests are distributed round-robin. In the *disaggregated* configuration there are prefill-only (P) and decode-only (D) nodes; x prefill and y decode nodes are written xPyD. On two nodes we compared two replicas with 1P1D; on three nodes, three replicas with 1P2D and 2P1D. Single nodes were used only for cross-GPU comparison.

3.4 Workloads

We used three workloads. *Chat* uses ShareGPT conversations [23] (median input about 80–100 tokens, output about 100–150). *Long input* uses synthetic requests with input fixed at 4,096, 16,384, or 32,000 tokens and output fixed at 512. *Real trace* replays 600 requests of the conversation trace from the Azure LLM inference dataset released with Splitwise (median input 996, output 217 tokens) [24]. Requests arrive as a Poisson process at a given mean rate; each run lasts 180 s.

3.5 Metrics and SLO

We judge each request individually. The *chat SLO* requires TTFT (time from sending the request to receiving the first output token) of at most 1 s and a per-request maximum inter-token latency (ITL, the time between two consecutive output tokens) of at most 50 ms. The *document SLO* uses 3 s and 250 ms. We use the per-request maximum gap because a single long pause in a 500-token answer is only 0.2% of its tokens and disappears in percentile statistics, yet it is exactly the stall a user notices.

Attainment is the fraction of requests that meet the SLO. *Supported load* is the highest request rate with attainment of at least 0.9; for points measured repeatedly, every repeat must reach 0.9. *SLO goodput* is the output-token throughput of SLO-meeting requests, and *cost* is the GPU hourly price divided by SLO goodput, in dollars per million tokens.

The 1 s TTFT threshold is a design choice. TTFT targets in prior work range from 0.125 s to 30 s depending on application and model (0.125 s for code completion and 0.25–4.0 s for chatbots in DistServe [1], 30 s in Mooncake's trace-replay experiment [3]). One second is the classic limit for a user's flow of thought to stay uninterrupted [25]. Section 4.6 checks how much the results depend on this choice.

3.6 Procedure and data selection

All measurements ran with the prefix cache disabled. Some early runs had it enabled while replaying the same seeded requests, so later points hit cache entries left by earlier points. We audited every run and excluded 122 such runs; together with runs that met discard rules and server start-up failures, we removed 128 of 389. Five runs where the client-side router hit its CPU limit are flagged as caution data. All results use the remaining 256 runs.

In the final, reliability-focused measurement we sent 30 warm-up requests to every server and measured key points three times with different seeds. Median TTFT moved by less than 5% across repeats and matched the previous day's measurement closely (Appendix Figure A5). Only points with attainment between 0.85 and 0.95 varied by up to ±0.09 across repeats, enough to flip the verdict; we mark such points as *borderline*. Appendix C maps measurement rounds to the released data folders.

4 Results

Section 4.1 fixes the evaluation method. Sections 4.2 and 4.3 answer RQ2, Sections 4.4 and 4.6 answer RQ3, and Sections 4.5 and 4.7 answer RQ1 and RQ4. Section 4.8 adds the real trace and a correctness check.

4.1 System percentiles hide per-request stalls

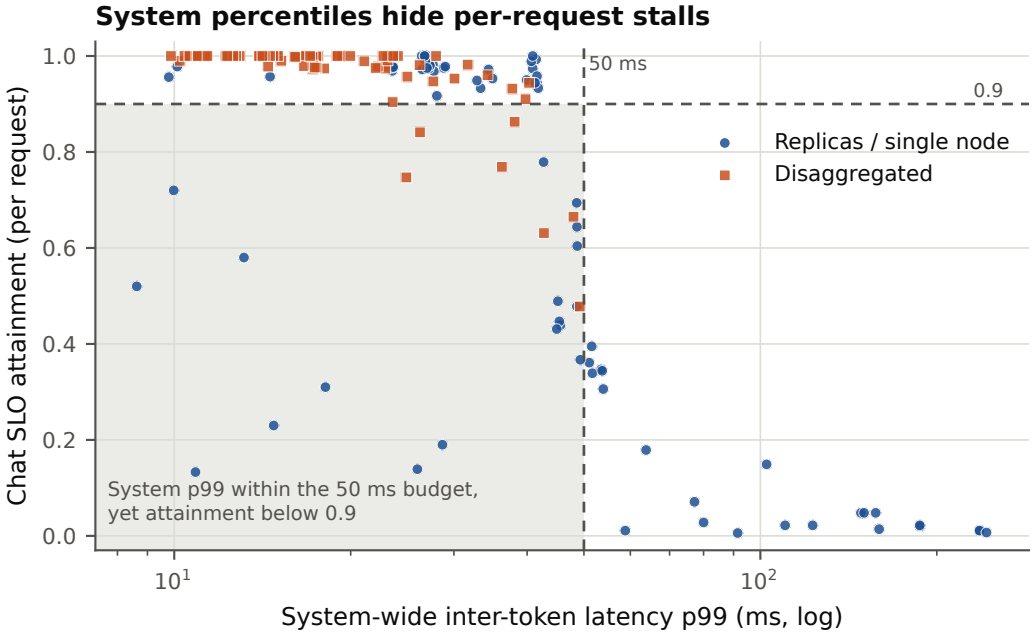


Figure 1: System-wide inter-token latency p99 (log scale) against per-request chat-SLO attainment for the 166 audited runs with TTFT p99 below 1 s. The shaded region holds runs whose system p99 is within the 50 ms budget yet whose attainment is below 0.9: 18 replica runs and 7 disaggregated runs.

The system-wide p99 of inter-token latency pools every gap of every request. It can look healthy while many requests still stall mid-response (Figure 1). Of the 166 runs with TTFT p99 below 1 s, 25 had system p99 at or below 50 ms but attainment below 0.9, 18 of them replicas. For example, two g7e replicas serving 26B FP8 with 4K inputs at 0.5 requests/s had a system p99 of 8.6 ms, yet 48% of requests had at least one gap over budget and attainment was 0.52. These stalls occur when another request's prefill is scheduled in between. All later verdicts are per request.

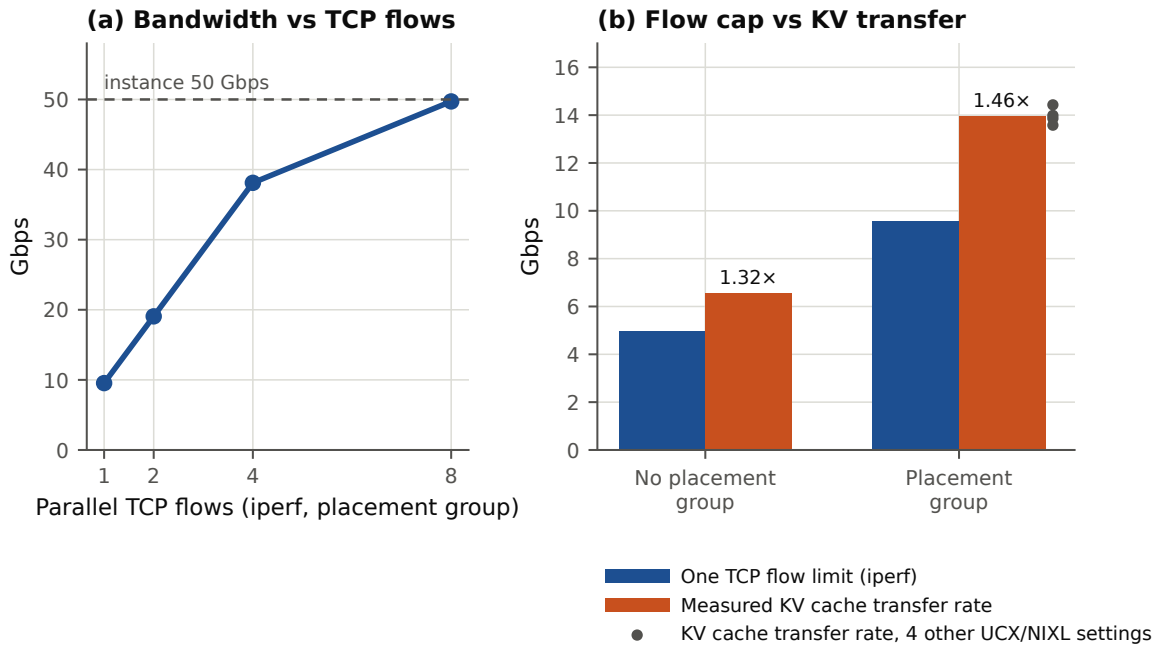


Figure 2: (a) Bandwidth between two g7e nodes in a cluster placement group versus the number of parallel TCP flows (iperf). (b) Two g7e nodes, 26B FP8, 4K input at 0.5 requests/s; outside and inside the placement group were measured on different days. Blue bars: one TCP flow's limit (iperf). Orange bars: the rate at which vLLM actually moved the KV cache (bytes / transfer time). Gray dots: KV transfer rate under four alternative settings (UCX segment size, NIXL thread count, fragment size, and so on), all close to the orange bar.

4.2 The transfer path: software first, then the per-flow cap

With default settings the bottleneck was software, not the network. UCX's default TCP segment of 8 KB limited the KV transfer; raising it to 256 KB doubled the rate from 3.0–4.05 to 7.07–8.49 Gbps. On two g7 nodes with 4K inputs, median TTFT at 0.25 requests/s fell from 626 to 466 ms, and chat attainment at 1 request/s rose from 0.82 to 0.99.

Next, the cloud's per-flow cap limited the transfer. A cluster placement group doubled the cap from 4.97 to 9.54 Gbps (Figure 2b, blue bars). The rate at which the KV cache actually moved followed the cap: 6.55 Gbps outside the placement group (1.32× the cap) and 13.96 Gbps inside it (1.46×; orange bars). It exceeds the cap slightly because NIXL splits a request's KV cache into several pieces carried on several connections; 31B, with more pieces, reached 1.68×. In contrast, a 1 MB segment, 16 NIXL threads, or larger rendezvous fragments all stayed within 13.58–14.43 Gbps and had no effect (gray dots). The instance's total bandwidth was still available: bandwidth grew almost linearly with the number of flows and reached the nominal 50 Gbps with eight (Figure 2a). The remaining headroom lies in the number of connections, not in settings.

Artificially limiting bandwidth shows that this cap sets TTFT (Appendix Figure A1). On two g7 nodes with 4K inputs at 0.5 requests/s, median TTFT was 471 ms without a limit, 744 ms at 3 Gbps, 954 ms at 2 Gbps, and 2,185 ms at 1 Gbps, with chat attainment 1.00, 0.92, 0.55, and 0.

Two things did not help or hurt. Staging the KV cache through host memory instead of GPU memory slowed TTFT by 4.7× at 0.25 requests/s and by 75× at 1 request/s. Network credits (burst allowances) were not a constraint: sending 4K inputs to an L40S pair for 15 minutes averaged 0.91 Gbps of transfer, 18% of the cap, and the single-flow bandwidth before and after the load was 4.967 and 4.966 Gbps.

4.3 The TTFT overhead is KV transfer time

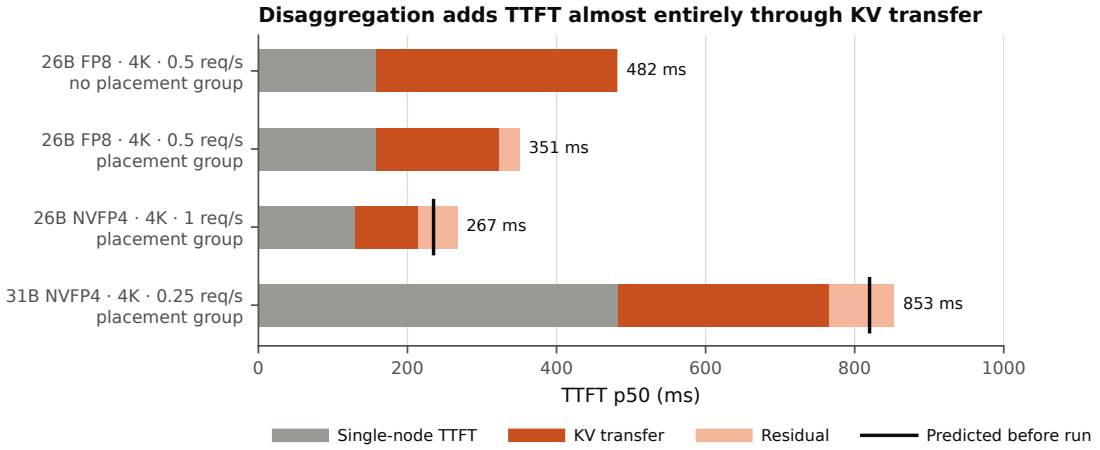


Figure 3: Median TTFT of disaggregated configurations split into single-node TTFT, KV transfer time (request mean), and residual. Black ticks are predictions written down with Eq. (1) before measuring (for 31B, the middle of the predicted 810–830 ms). The top two bars were measured on different days, and the first bar's transfer time aggregates only part of the requests, so it is indicative only.

The TTFT overhead of disaggregation is almost entirely time spent waiting for the KV transfer. On two g7e nodes serving 26B FP8 with 4K inputs at 0.5 requests/s, doubling the per-flow cap with a placement group cut the mean transfer time from 324 to 165 ms and median TTFT from

482 to 351 ms (Figure 3); the two reductions are nearly equal. TTFT without transfer (351 − 165 = 186 ms) is close to the single-node TTFT of 158 ms at the same point. These two measurements were taken on different days, and the transfer time outside the placement group aggregates only part of the requests, so the main validation is the prediction below.

The relation can be written as

$$\mathrm{TTFT}_{\mathrm{PD}} \approx \mathrm{TTFT}_{\mathrm{single}} + \frac{\text{input tokens} \times \text{bytes per input token}}{\text{per-flow cap} \times m} + \text{residual}, \tag{1}$$

where m (1.3–1.7) reflects spreading the transfer over several connections, and the residual is the first decode step plus scheduling. We wrote this model down before the three-node measurements and compared. The transfer volume matched the prediction exactly: 140 MB for 26B NVFP4 and 560 MB for 31B NVFP4. For 26B at 4K and 1 request/s, TTFT was predicted at 235 ms and measured at 267 ms (single 130 + transfer 84 + residual 53); for 31B at 4K and 0.25 requests/s, predicted 810–830 ms and measured 853 ms (single 483 + transfer 283 + residual 87). The larger residual for 31B is likely due to its longer first decode step (25 ms versus 6.5 ms for 26B).

With this formula one can check before deploying whether the transfer fits the TTFT budget. For example, where the per-flow cap is about 5 Gbps, a 16K input to 26B FP8 needs more than a second for the transfer alone and cannot meet a 1 s chat TTFT budget.

4.4 When disaggregation wins on two nodes

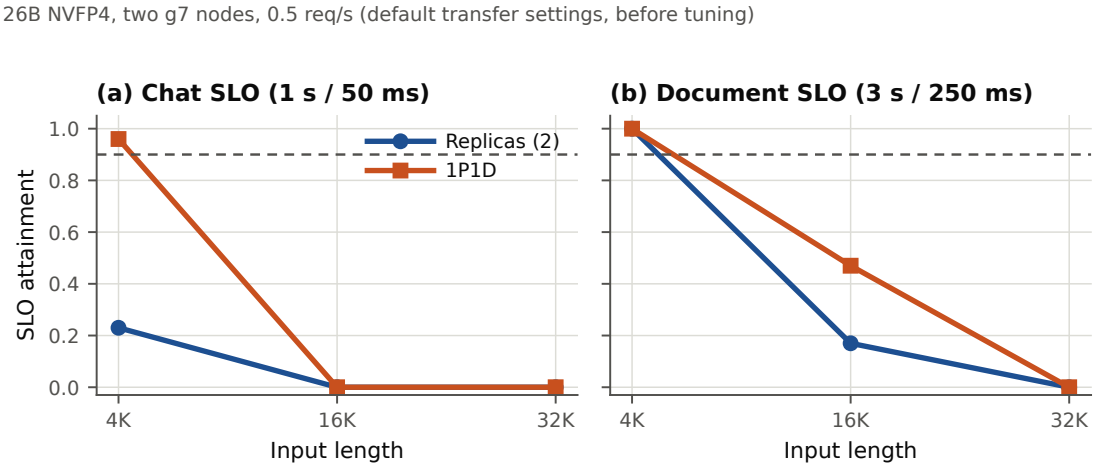


Figure 4: Attainment versus input length on two g7 nodes, 26B NVFP4, 0.5 requests/s, measured before transfer tuning (UCX defaults). (a) Chat SLO. (b) Document SLO.

Input length decides. On two g7 nodes serving 26B NVFP4 at 0.5 requests/s with 4K inputs, chat attainment was 0.96 for 1P1D and 0.23 for replicas (Figure 4a). Replicas failed on inter-token gaps, not on TTFT. This run predates transfer tuning, so disaggregation was at a disadvantage and still led by a wide margin. From 16K on, replicas exceeded 1 s of TTFT from prefill alone and disaggregation added transfer on top, so neither met the chat SLO. Under the document SLO, disaggregation still led at 16K (0.47 versus 0.17, Figure 4b). At 32K all configurations failed. For chat-length inputs, measurements on other GPUs showed either no difference or replicas doing better (Sections 4.5 and 4.6).

Faster GPUs narrow the gap. At 4K inputs and 1 request/s, replicas collapsed to 0.02–0.14 attainment on RTX PRO 4500, L40S, and RTX PRO 6000 (FP8), while disaggregation reached 0.83–0.98 (Appendix Figure A2; the g7 measurement predates transfer tuning). A faster GPU has shorter decode steps and can absorb more prefill interference. With lighter NVFP4 weights and a placement group on RTX PRO 6000, even replicas held 0.97.

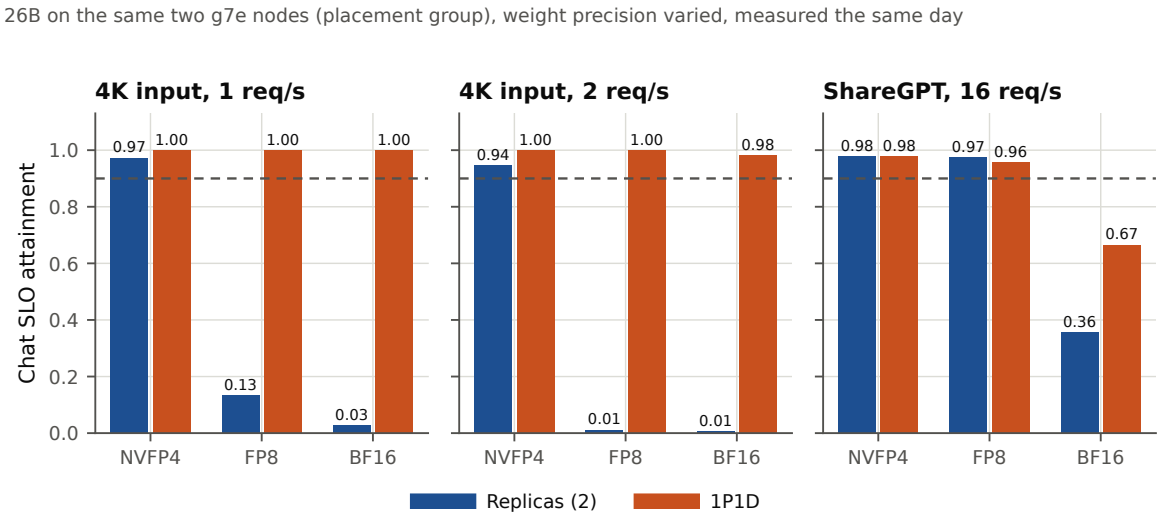


Figure 5: Weight precision varied on the same two g7e nodes in a cluster placement group, 26B, measured the same day.

Weight precision changes the size of the gain. Varying only the precision on the same two nodes (Figure 5), replica attainment at 4K and 1 request/s was 0.97 with NVFP4, 0.13 with FP8, and 0.03 with BF16, while disaggregation reached 1.00 with all three. Every replica failure was an inter-token-gap failure; TTFT p99 stayed below 1 s. The heavier the weights, the longer one 4K prefill stalls the decode. With FP8 and BF16, disaggregation is effectively the only option; with NVFP4 replicas cope to a degree and the gap narrows. For chat at 16 requests/s, disaggregation brought no gain at any precision, and with BF16 both approaches failed.

Disaggregated TTFT grew with KV size: 269 ms (NVFP4), 362 ms (FP8), and 448 ms (BF16) at 4K and 1 request/s. The FP8 and BF16 checkpoints keep a BF16 KV cache and transfer twice as much; differences in prefill speed are mixed in, and we did not isolate the transfer share.

4.5 Scaling to three nodes and the ratio rule

26B, 4K input: disaggregation raises per-GPU capacity. Replicas failed at 2 requests/s per GPU (two replicas: 0.37–0.64 in four runs; three replicas: 0.69). 1P1D reached 1.00 at 3 requests/s per GPU (6 requests/s) and 0.84 at 4 (Figure 6a). Comparing the loads that passed, the gain is 3×; conservatively it is at least 1.5×. Three nodes did not improve per-GPU efficiency beyond 1P1D: 1P2D was borderline at 3 requests/s per GPU (0.906 at 9 requests/s) and at 12 requests/s its prefill queue grew until median TTFT reached 14.3 s, while 2P1D dropped to 0.77 at 9 requests/s because its single decode node could not hold the gap limit. For 26B with 4K inputs, one prefill and one decode node carry similar loads, so 1:1 is balanced.

Figure 6: 26B NVFP4 on g7e in a cluster placement group; the x-axis is request rate per GPU. (a) 4K input. (b) ShareGPT chat. Vertical bars span the minimum and maximum of repeated runs (including the previous day). Hollow markers are client-limited points and are not used for verdicts.

26B, chat: no gain. At 8 requests/s per GPU all configurations passed (Figure 6b). Higher loads hit the client router's limit and were excluded from verdicts. Server metrics explained why 2P1D collapsed there: its single decode node hit the concurrent-sequence limit of 256, the queue grew to 868 requests, and KV usage was only 43%. Chat needs more decode capacity.

Figure 7: 31B NVFP4 on g7e in a cluster placement group; the x-axis is request rate per GPU. (a) ShareGPT chat. (b) 4K input. Vertical bars span the minimum and maximum of repeated runs (including the previous day).

31B, chat: only disaggregation meets the SLO. Two replicas stayed at 0.43–0.49 in all four runs even at 2 requests/s (Figure 7a). The dense model's decode step is long enough that even

short prefills push gaps over 50 ms. 1P1D passed at 4 requests/s (0.98), and 8 requests/s was borderline (0.863–0.944 over four runs).

31B, 4K input: no configuration holds the SLO reliably. Replicas failed on gaps and disaggregation on TTFT (Figure 7b). One 31B prefill of 4K tokens takes about 480 ms and the transfer adds about 280 ms, leaving little of the 1 s budget. 1P1D at 0.25 requests/s and 2P1D at 0.75 requests/s each exceeded 0.9 in only one of four runs. At 0.75 requests/s, chat goodput was 290 tokens/s for 2P1D, 177 for 1P2D, and 8 for three replicas: long-input 31B needs more prefill capacity.

In short, the ratio should grow the bottleneck side: 1:1 for 26B with 4K inputs, more prefill for 31B with 4K inputs, and more decode for 26B chat.

4.6 How much the verdict depends on the SLO

Because every per-request timing is stored, the same runs can be re-judged under other SLOs. For each combination of TTFT limit (0.25–5 s) and maximum-gap limit (30–300 ms) we compared the per-GPU SLO goodput of the two approaches at the highest rate where every repeat reaches 0.9 attainment (Figure 8; ratios in Appendix Figure A7).

Disaggregation wins where mid-response stalls must be tightly bounded, roughly with a maximum-gap limit of 50–75 ms or less. For 26B with 4K inputs (g7e, placement group), only disaggregation passed at 40 ms or below; it served 3.0× more at 50 ms, and at 75 ms or more the ratio fell to 2× or less. For 31B chat, disaggregation alone passed only at 40–75 ms; at 100 ms or more both approaches passed up to the highest measured rate. For 31B with 4K inputs, both failed with a 1 s TTFT limit, but at 1.5 s only disaggregation passed. 16K inputs required relaxing the TTFT limit to 2 s (g7e FP8) or 5 s (g7) before disaggregation alone passed. For 26B chat in the placement group, both approaches passed up to the highest measured rate and could not be separated. In contrast, for FP8 chat on g7e, replicas were better under every SLO (0.3–0.7×).

Looking at latency and throughput without an SLO explains why (Figure 9). Replicas always have a faster first token. The per-request maximum gap is shorter under disaggregation for 4K inputs and for 31B; for 26B chat, replicas also keep gaps near 30 ms, so disaggregation has nothing to offer. For 26B at 4K, the replicas' p90 maximum gap sits at 50–65 ms, right on the 50 ms line, which makes the verdict at that point sensitive to the 50 ms choice. For 31B chat the replicas' p90 maximum gap is 70–90 ms versus 35–48 ms for disaggregation. Disaggregation trades first-token latency for a steady token stream, and the value of that trade depends on how much the service dislikes stalls.

4.7 Cost

Workload	Replicas	Disaggregated
26B, 4K, two nodes	\$1.98 (2 req/s)	\$0.63 (1P1D, 6 req/s)
26B, 4K, three nodes	\$1.89 (3 req/s)	\$0.69 (1P2D, 9 req/s, borderline)
26B, chat, two nodes	\$0.63 (16 req/s)	\$0.63 (16 req/s)
31B, chat, two nodes	no supported load	\$2.58 (4 req/s)
31B, 4K, two nodes	no supported load	no supported load

Table 2: Cost per million SLO-meeting output tokens, at the highest request rate where every repeat met the chat SLO (g7e at \$3.363 per GPU-hour, client excluded).

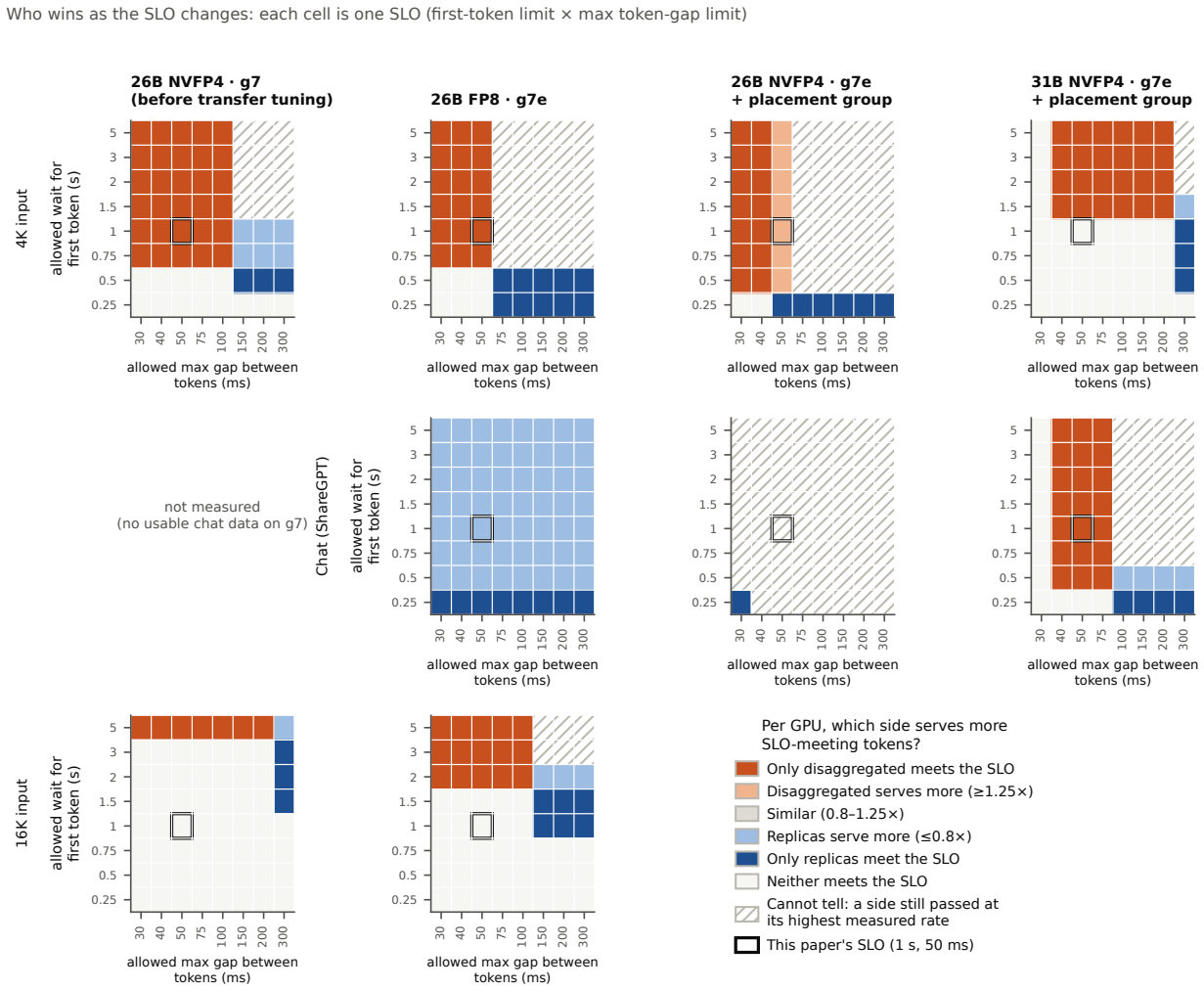


Figure 8: Which approach serves more SLO-meeting tokens per GPU as the SLO changes. Each cell is one SLO (vertical: allowed TTFT; horizontal: allowed per-request maximum inter-token gap). Columns are setups and rows are workloads. Orange cells favor disaggregation and blue cells favor replicas; dark cells mean only one side meets the SLO. Hatched cells cannot be decided because one or both sides still passed at their highest measured rate; very light cells mean neither side meets the SLO. The bold outline marks this paper's SLO (1 s, 50 ms). Per-cell ratios are in Appendix Figure A7.

Latency and throughput without an SLO, g7e in a placement group (lower-right is better). Dashed: 1 s and 50 ms. Hollow: client-limited.

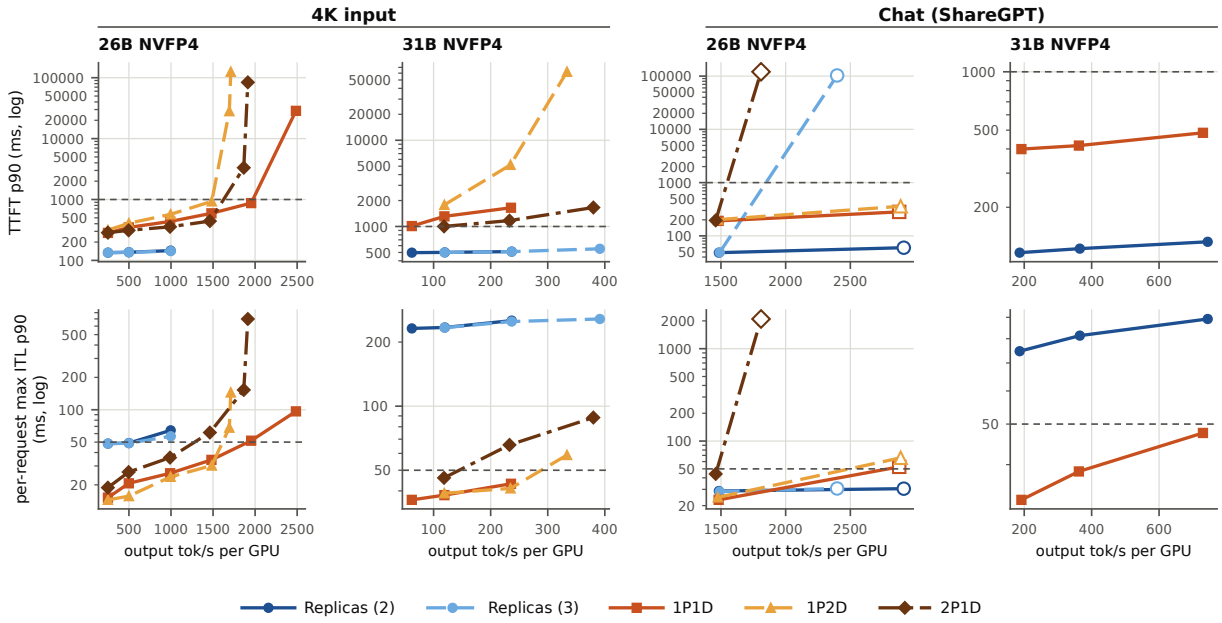


Figure 9: Latency and throughput without an SLO (g7e in a cluster placement group). The left two columns are 4K input and the right two are chat, each in the order 26B, 31B. The x-axis is output tokens/s per GPU; the top row is per-request TTFT p90 and the bottom row is the p90 of the per-request maximum inter-token gap. Dashed lines mark 1 s and 50 ms; hollow markers are client-limited.

Measured per SLO-meeting token, disaggregation is about 3× cheaper for 26B with 4K inputs (Table 2). For 26B chat the cost is equal, and for 31B chat only the disaggregated configuration has a cost at all.

GPU differences also grow once an SLO is applied (Appendix Figure A3). With the same 26B FP8 model and identical KV capacity, the supported chat load on a single node was 1 request/s on L40S and 16 on RTX PRO 6000, and the SLO cost differed by 9.2× ($3.15 versus $0.34). The two values were measured on different days.

4.8 Real trace, correctness, and push transfer

We replayed the Azure conversation trace for 26B on two nodes (about 4 requests/s) and three nodes (about 5.9 requests/s), and for 31B on two nodes (about 3.7 requests/s) (Appendix Figure A4). For 26B every configuration passed at equal cost; the only difference was TTFT, where disaggregation was about 120 ms slower, the transfer time. This load was below every 26B configuration's limit and does not rank them. For 31B both approaches failed: replicas on gaps (attainment 0.05) and disaggregation on the TTFT tail (0.28).

We also checked that disaggregation carries context correctly. We sent six short chat-formatted prompts and two prompts of about 4K tokens at temperature 0, and compared whether short prompts produced the same output as a single node and whether long prompts shared enough content words.

The push mode, where prefill writes the KV cache instead of decode reading it, cut transfer time by 27–36% and median TTFT by 53–62 ms for 31B at 4K, but attainment changed by only ±0.04 (Appendix Figure A6). Within a single node over PCIe, push has been reported to be about 100× faster than pull [26]; we saw no such difference across nodes over TCP. A proposed vLLM patch that overlaps KV transfer with prefill layer by layer (PR #56874[1]) supports only models with a single KV-cache group and stalled on the first request with hybrid-attention Gemma 4.

5 Discussion

5.1 Answers to the research questions

RQ1. Yes, conditionally. For 26B with 4K inputs, disaggregation raised the per-GPU supported load by 1.5–4× and cut the cost per SLO-meeting token by about 3×. For dense 31B chat, only disaggregation met the SLO. For 26B chat it brought no gain (Sections 4.5 and 4.7).

RQ2. The KV transfer time. It is proportional to input tokens and bytes per input token and inversely proportional to the effective bandwidth (Eq. (1)). The effective bandwidth is set first by transfer-software settings and, once those are fixed, by the cloud's per-flow cap (Sections 4.2 and 4.3).

RQ3. Two conditions must hold together: the per-request maximum gap that prefill interference causes in replicas must exceed the service's tolerance, and the transfer time added by disaggregation must fit within the TTFT budget. Longer inputs, heavier weights, and dense models make the first condition easier to meet. With a tolerance of 100 ms or more the difference mostly disappears (Sections 4.4 and 4.6).

[1] `https://github.com/vllm-project/vllm/pull/56874`

RQ4. Grow the bottleneck side. For 26B with 4K inputs 1:1 was most efficient; 31B with 4K inputs needed more prefill and 26B chat more decode (Section 4.5).

5.2 A checklist before disaggregating

Turning the measurements into a deployment decision gives four checks.

1. **Stall tolerance versus prefill interference.** Does the replicas' per-request maximum gap exceed the service's tolerance? If not, disaggregation only delays the first token. Heavier weights, dense models, and longer inputs make it more likely.
2. **Transfer headroom.** Does input tokens $\times$ bytes per input token $\div$ effective bandwidth fit within the TTFT budget (Eq. (1))? The cloud's per-flow cap and placement group decide the effective bandwidth.
3. **Transfer-software settings.** Fix defaults that are bottlenecks first, such as UCX's TCP segment size. Beyond that, headroom lies in the number of connections rather than in settings.
4. **Ratio.** Grow the bottleneck side: prefill for large models with long inputs, decode for short-input chat.

With RDMA or AWS EFA, the per-flow cap and m in Eq. (1) change substantially and the transfer term shrinks, which should widen the region where disaggregation wins; we did not measure that baseline. Going beyond the per-flow cap requires spreading the transfer across more connections; UCCL, reported by llm-d, is one example [8].

6 Limitations

- One model family (Gemma 4), both with hybrid attention. Models with only global attention have larger per-token KV caches and a larger transfer term.
- One vLLM version (v0.30.0), one availability zone in one AWS region, and at most three nodes.
- No RDMA baseline; we also did not measure slow links inside a node (PCIe without NVLink or P2P).
- Coarse request-rate grids; under relaxed SLOs the replicas' limit lies beyond the measured range.
- No prompt reuse. Workloads with many prefix-cache hits (multi-turn chat, shared system prompts) reduce prefill and transfer and may change the conclusions.
- Latency was measured at the client and includes the router. Some high-load chat points hit the client limit and were excluded from verdicts.
- Some comparisons (the KV-matched cross-GPU comparison) join measurements from different days.

7 Conclusion

Even over slow TCP links, disaggregated serving can raise SLO goodput, conditionally. Its TTFT overhead is the KV transfer time and can be computed in advance. Disaggregation pays off when

the service must tightly bound mid-response stalls, when long inputs or heavy models make prefill interference large, and when the transfer time for those inputs fits within the TTFT budget. These conditions can be judged before measuring from four numbers: input length, KV bytes per token, per-connection bandwidth, and stall tolerance.

Use of AI tools

An AI coding assistant (Claude) was used to write measurement and analysis scripts, operate the cloud infrastructure, draft figures, and draft and translate the text. All experiments ran on real hardware, and all numbers were checked by the author against the raw data.

Acknowledgments

This work was carried out at the author's own expense (about $220 of cloud usage).

References

[1] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.

[2] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024. doi: 10.1109/ISCA59077.2024.00019.

[3] Ruoyu Qin, Zheming Li, Weiran He, et al. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, 2025.

[4] llm-d project. Disaggregated serving (documentation, v0.9). `https://llm-d.ai/docs/architecture/advanced/disaggregation`, 2026.

[5] NVIDIA. Disaggregated inference communication guide (Dynamo v1.3.0 documentation). `https://docs.nvidia.com/dynamo/v1.3.0/kubernetes-deployment/operate/disagg-communication`, 2026.

[6] Ruoyu Qin, Weiran He, Yaoyu Wang, Zheming Li, et al. Prefill-as-a-service: KVCache of next-generation models could go cross-datacenter, 2026.

[7] Yida Wang, Xiuhong Li, Jianping Ma, Gan Sun, Yunshen Xu, et al. PDD: Unleashing economical and flexible heterogeneous LLM inference via cross-datacenter prefill-decode disaggregation, 2026.

[8] Pravein Govindan Kannan, Liran Schour, Aleksander Slominski, Raj Joshi, Nicolò Lucchesi, Carlos Costa, Moein Khazraee, and Omri Kahalon. Networking for distributed inference in llm-d. llm-d blog, `https://llm-d.ai/blog/networking-for-distributed-inference-llm-d`, 2026. June 23, 2026.

[9] Cunchen Hu, Heyang Huang, Liangliang Xu, et al. Inference without interference: Disaggregate LLM inference for mixed downstream workloads, 2024.

[10] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, 2024.

[11] Yibo Jin, Tao Wang, Huimin Lin, et al. P/D-Serve: Serving disaggregated large language model at scale, 2024.

[12] Amey Agrawal et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.

[13] Ke Hong et al. semi-PD: Towards efficient LLM serving via phase-wise disaggregated computation and unified storage, 2025.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.

[15] NVIDIA. NIXL: NVIDIA inference xfer library. `https://github.com/ai-dynamo/nixl`, 2026.

[16] Pavel Shamis, Manjunath Gorentla Venkata, M. Graham Lopez, et al. UCX: An open source framework for HPC network APIs and beyond. In *IEEE 23rd Annual Symposium on High-Performance Interconnects (HOTI)*, pages 40–43, 2015. doi: 10.1109/HOTI.2015.13.

[17] Yuhan Liu, Hanchen Li, Yihua Cheng, et al. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024. doi: 10.1145/3651890.3672274.

[18] Tiyasa Mitra et al. Beyond the buzz: A pragmatic take on inference disaggregation, 2025.

[19] vLLM project. Disaggregated prefilling (experimental). `https://docs.vllm.ai/en/latest/features/disagg_prefill/`, 2026.

[20] Amazon Web Services. Amazon EC2 instance network bandwidth (Amazon EC2 user guide). `https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-network-bandwidth.html`, 2026.

[21] Google. Gemma 4 model card. `https://ai.google.dev/gemma/docs/core/model_card_4`, 2026.

[22] NVIDIA. Gemma-4-26B-A4B-NVFP4 and Gemma-4-31B-IT-NVFP4 model checkpoints. `https://huggingface.co/nvidia/Gemma-4-26B-A4B-NVFP4`, `https://huggingface.co/nvidia/Gemma-4-31B-IT-NVFP4`, 2026.

[23] ShareGPT_Vicuna_unfiltered: ShareGPT_V3_unfiltered_cleaned_split.json. `https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered`, 2023.

[24] Microsoft Azure. Azure LLM inference trace 2023 (AzurePublicDataset). `https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md`, 2023.

[25] Jakob Nielsen. *Usability Engineering*. Academic Press, 1993. Chapter 5; summarized in "Response Times: The 3 Important Limits", Nielsen Norman Group.

[26] vLLM project. Issue #52607: vLLM's default NIXL connector is ~100× slower than the push connector on non-RDMA hardware. `https://github.com/vllm-project/vllm/issues/52607`, 2026.

A Additional figures

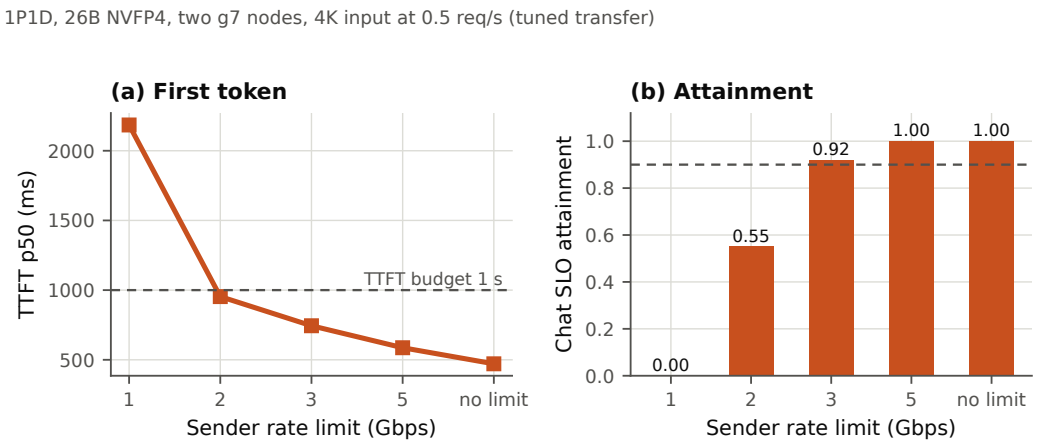


Figure A1: 1P1D on two g7 nodes, 26B NVFP4, 4K input at 0.5 requests/s, with the sender's bandwidth limited. (a) Median TTFT. (b) Chat attainment. Measured after transfer tuning.

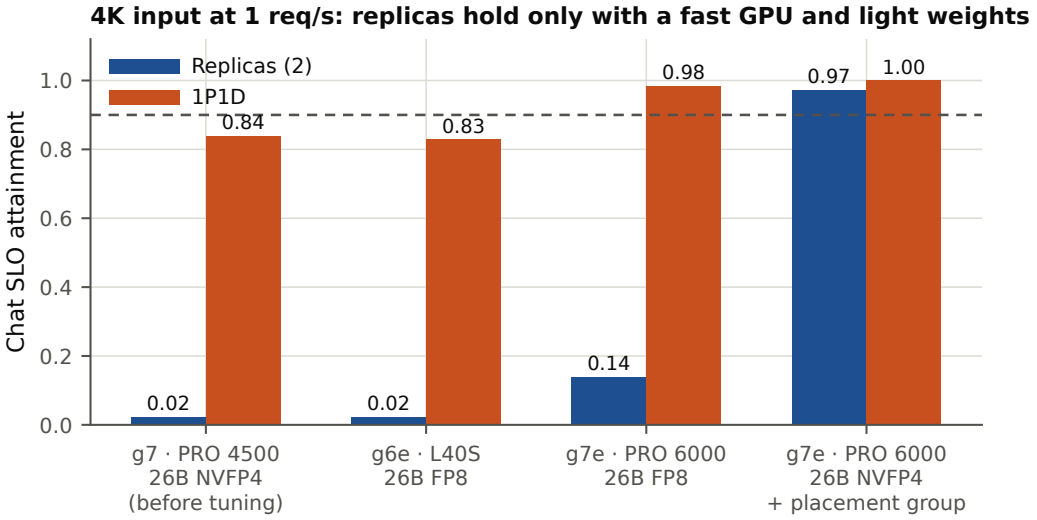


Figure A2: Chat attainment of two-node configurations at 4K input and 1 request/s by GPU and precision. The g7 measurement predates transfer tuning, which disadvantages disaggregation.

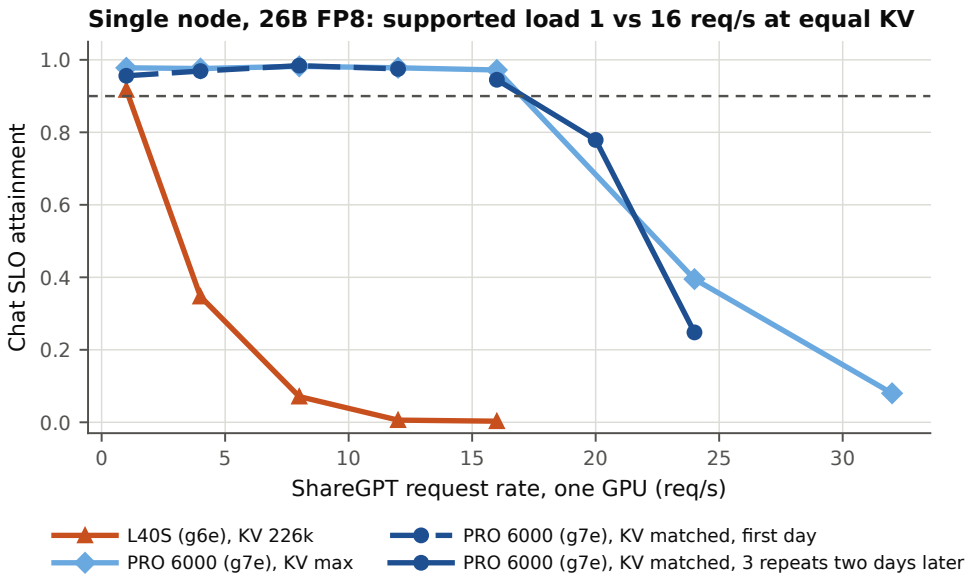


Figure A3: Chat attainment of a single node, 26B FP8. RTX PRO 6000 with KV capacity matched to L40S (dashed: first day; solid: three repeats two days later) and with maximum KV capacity.

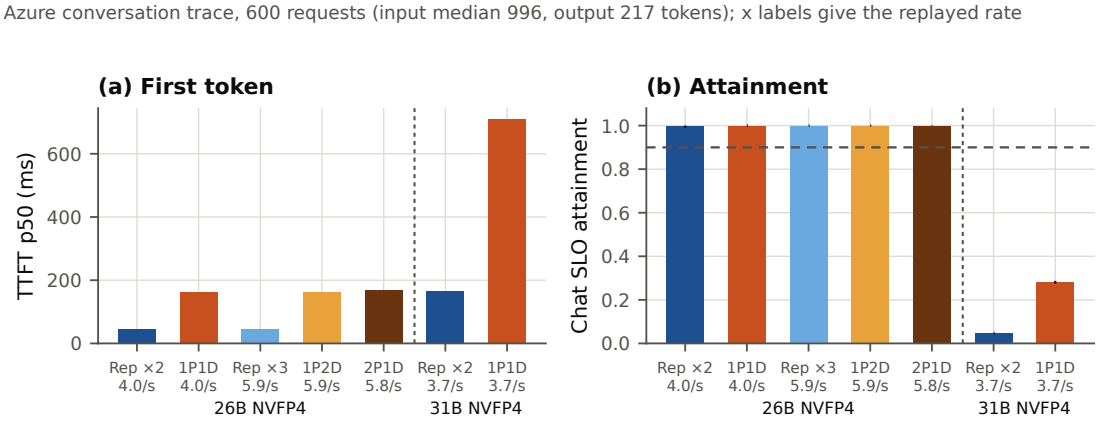


Figure A4: Azure conversation trace, 600 requests. (a) Median TTFT. (b) Chat attainment. Numbers under the x-axis give the replayed rate; left of the dotted line is 26B, right is 31B.

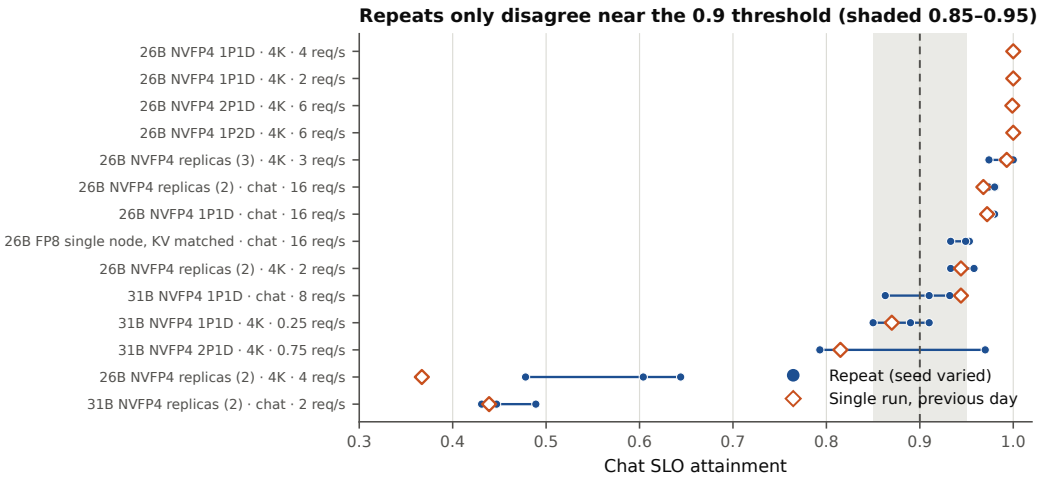


Figure A5: Attainment at points measured at least three times with different seeds. Blue dots are repeats; orange diamonds are the previous day's run. The gray band is 0.85–0.95. In the top four rows all repeats and the previous run are 1.00, so the dots are hidden behind the diamonds.

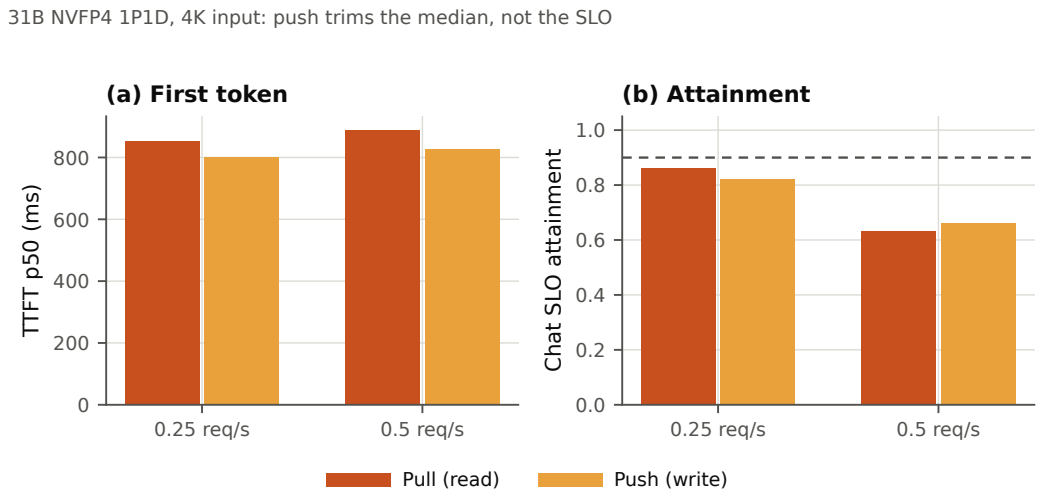


Figure A6: 31B NVFP4 1P1D, 4K input, pull versus push transfer. (a) Median TTFT. (b) Chat attainment; the dashed line is 0.9.

Ratio behind figure 8: SLO goodput per GPU at the highest rate where every repeat reaches 0.9, disaggregated ÷ replicas
(best config of each kind). Orange: disaggregation wins. ≥/≤: one side still passing at its highest measured rate. ?: both. D or R: only that side reaches 0.9. Box: 1 s / 50 ms.

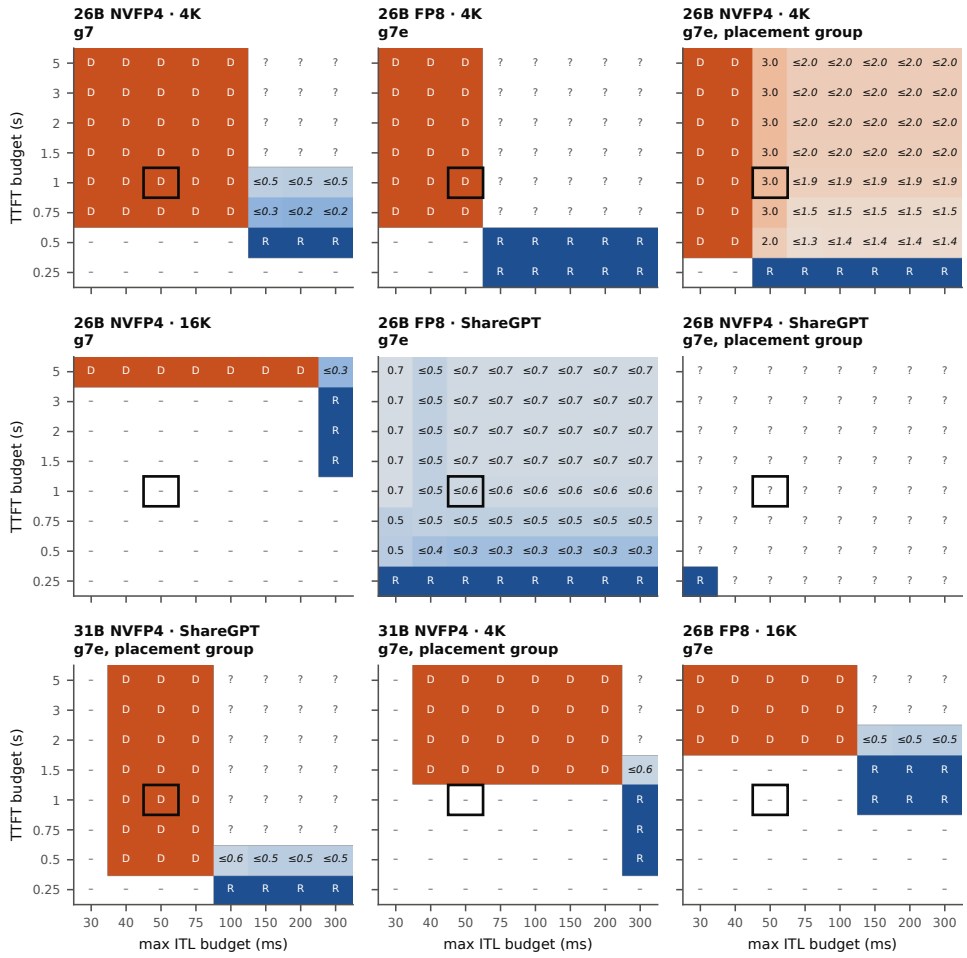


Figure A7: Per-cell ratios behind Figure 8: per-GPU SLO goodput of disaggregation divided by that of replicas. D and R mark cells where only that side reaches 0.9; ≤ and ≥ mark cells where one side still passed at its highest measured rate, so the true ratio may be smaller or larger; ? marks cells where both did; – marks cells where both failed.

B Practical pitfalls

- If UCX's TCP receive segment is smaller than the send segment, the engine refuses to start; raise both together.
- At high request rates the request router hit the operating system's file-handle limit (1,024 by default) and requests failed. A disaggregated router opens two connections per request and hits the limit sooner.
- In vLLM v0.30.0, trace replay without an explicit request count builds an empty request list and fails.
- Replaying the same seeded requests against a server with prefix caching enabled makes later points hit the cache left by earlier points and inflates results.

C Data and code

We release the measurement harness, configurations, analysis scripts, and per-request raw data [GitHub URL and Zenodo DOI]. Raw data are stored per measurement round (`results/<folder>/`); Table C1 maps them to the results.

Folder	Date (UTC)	Instances	Models	Main content (sections)
B	2026-09-25	2×g7	26B NVFP4	input-length grid (4.4)
D	2026-09-26	2×g7	26B NVFP4	transfer tuning, bandwidth limit (4.2)
E1	2026-09-26	1×g6e, 1×g7e	26B, 31B FP8	cross-GPU, KV matching (4.7)
E2	2026-09-26	2×g7e	26B FP8	two-node PD vs. replicas (4.4, 4.6)
E3	2026-09-27	2×g6e	26B FP8	L40S pair, network credits (4.2, 4.4)
E4	2026-09-27	2×g7e, PG	26B FP8	placement group, transfer settings (4.2, 4.3)
E5	2026-09-27	2–3×g7e, PG	26B, 31B NVFP4	scaling, transfer prediction (4.3, 4.5)
E6	2026-09-27	2×g7e, PG	26B, 31B NVFP4	push transfer, correctness (4.8)
E7	2026-09-28	2–3×g7e, PG	26B NVFP4/FP8/BF16, 31B NVFP4	repeats, trace, precision, KV matching (3.6, 4.4–4.8)

Table C1: Measurement rounds and data folders (PG: cluster placement group). Folders A, A′, C1, and P are prefix-cache-contaminated or plumbing runs and are not used.